Guía de Actividades **Práctico-Experimentales** Nro. 004

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Integrantes	Matías Encarnación Matias Lavanda Julián Merino Jefferson Sarango Sherman Abarca
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Implementa algoritmos de abstracción mediante el diseño lógico y físico de Tipos de Datos Abstractos (TDA), asegurando eficiencia en la gestión de memoria.
Práctica Nro.	004
Título de la Práctica	Listas Dinámicas Simples sin Overhead y su Impacto en la CPU.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 22 de 10:30 – 11:30 Viernes 23 de mayo 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- **Diseñar e implementar** el Tipo de Dato Abstracto (TDA) Lista Simple Enlazada mediante la instanciación dinámica de nodos en JavaScript, evitando el uso de arreglos nativos.
- **Cuantificar y contrastar** el tiempo de ejecución T(n) entre la inserción de elementos al inicio de un arreglo dinámico (operación *unshift*) versus la inserción en la cabeza de una lista enlazada (operación *prepend*).
- **Evaluar** los principios de *Green Computing*, comprendiendo cómo la reducción de la complejidad temporal de ***O(n)*** a ***O(1)*** disminuye los ciclos de reloj de la CPU, mitigando el consumo energético y térmico del hardware.

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Se aplica ABP para resolver un problema de consistencia de datos en la transferencia de pacientes entre médicos.

Contexto para el estudiante: Imagina que trabajas para el equipo de ingeniería de una nueva red social. El servidor debe procesar millones de peticiones de "siguiente video" por segundo. Si usas una estructura ineficiente, el servidor consumirá más electricidad, aumentará su temperatura y elevará los costos operativos.

Para resolver esto, implementaremos una Lista Doblemente Enlazada que permita saltar entre videos hacia adelante y hacia atrás sin tener que recorrer toda la lista desde el principio.

Tarea 1: Creación del TDA Nodo y Lista Simple
Empezaremos construyendo la unidad básica de memoria dinámica: el Nodo.

1. En su directorio de trabajo, cree un archivo llamado lab04_lista_simple.js.
2. Implemente las clases Nodo y ListaSatelital utilizando clases modernas de JavaScript (ES6).

```
// Tarea 1: Implementación del TDA Lista Enlazada
class Nodo {
    constructor(dato) {
```

```
        this.dato = dato; // Carga útil (la coordenada)
        this.siguiente = null; // Puntero al siguiente nodo (referencia)
    }
}

class ListaSatelital {
    constructor() {
        this.cabeza = null; // Primer elemento de la lista
        this.tamaño = 0;    // Contador para S(n)
    }

    // Método para insertar al inicio: Complejidad O(1)
    prepend(dato) {
        const nuevoNodo = new Nodo(dato);
        if (!this.cabeza) {
            this.cabeza = nuevoNodo;
        } else {
            nuevoNodo.siguiente = this.cabeza;
            this.cabeza = nuevoNodo;
        }
        this.tamaño++;
    }
}

module.exports = { ListaSatelital };
```

Tarea 2: Simulación del Estrés Térmico (Arreglo Nativo vs Lista Enlazada)

Vamos a demostrar empíricamente por qué la Lista Enlazada es una estructura de datos sostenible (Green Computing) frente a inserciones masivas.

1. Cree un archivo llamado lab04_benchmark.js e importe la clase creada.
2. Implemente un script que mida los milisegundos requeridos para insertar N = 100,000$coordenadas al inicio de ambas estructuras.

```
const { performance } = require('perf_hooks');
const { ListaSatelital } = require('./lab04_lista_simple');

const N = 100000; // 100 mil registros satelitales

console.log(`[Benchmark] Iniciando inserción de ${N} elementos al INICIO...`);

// ------------------------------------------------------------
// PRUEBA 1: ARREGLO NATIVO (Desplazamiento de memoria O(n))
// ------------------------------------------------------------
let arregloHistorial = [];
const startArray = performance.now();

for (let i = 0; i < N; i++) {
    // unshift() fuerza a V8 a mover los índices de todos los elementos previos
```

```
    arregloHistorial.unshift({ id: i, lat: 10.5, lng: -79.2 });
}

const endArray = performance.now();
const timeArray = (endArray - startArray).toFixed(2);
console.log(`[Energía Alta] Arreglo nativo (unshift): ${timeArray} ms`);

// ------------------------------------------------------
// PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
// ------------------------------------------------------
let listaHistorial = new ListaSatelital();
const startList = performance.now();

for (let i = 0; i < N; i++) {
    // prepend() solo reasigna un puntero, sin mover elementos existentes
    listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
}

const endList = performance.now();
const timeList = (endList - startList).toFixed(2);
console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);

// Conclusión de impacto
const multiplicador = (timeArray / timeList).toFixed(1);
console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces
más rápida.`);
console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el
servidor.`);
```

3. Ejecute el script con node lab04_benchmark.js.

4. Análisis: Documente los tiempos arrojados. Notará que la diferencia es abismal. Mientras que el arreglo presenta una degradación asintótica $O(n^2)$ en el bucle completo, la lista mantiene un crecimiento lineal $O(n)$.

Tarea 3: Implementación del Recorrido (Traversal)

A diferencia de un arreglo, no podemos acceder a un nodo de la lista mediante índices (ej. lista[5]). Debemos recorrer la lista.

1. Regrese al archivo lab04_lista_simple.js y agregue el siguiente método dentro de la clase ListaSatelital.

```
    // Método para imprimir la lista (Recorrido secuencial O(n))
    imprimirPrimeros(limite) {
        let actual = this.cabeza;
        let contador = 0;
        let salida = "";

        while (actual && contador < limite) {
            salida += `[ID: ${actual.dato.id}] -> `;
```

```
            actual = actual.siguiente;
            contador++;
        }
        console.log(salida + "NULL");
    }
```

2. Pruebe el método en su archivo benchmark.js añadiendo al final: listaHistorial.imprimirPrimeros(5); para verificar la correcta vinculación de los nodos.

6. Resultados Esperados

El estudiante evidenciará a través del profiler (rendimiento en ms) que el uso de abstracciones de datos correctas mitiga drásticamente la latencia del software. Al reemplazar una estructura de datos de bloques contiguos (Arreglo) por una dispersa conectada por punteros (Lista Enlazada), se demuestra el ahorro de cálculos de la Unidad Aritmético Lógica (ALU) al evitar los redimensionamientos de memoria.

Código inicial de lab04_benchmark.js

```
const { performance } = require('perf_hooks');
const { ListaSatelital } = require('./lab04_lista_simple');

const N = 100000; // 100 mil registros satelitales

console.log(`[Benchmark] Iniciando inserción de ${N} elementos al INICIO...`);

// ------------------------------------------------------
// PRUEBA 1: ARREGLO NATIVO (Desplazamiento de memoria O(n))
// ------------------------------------------------------
let arregloHistorial = [];
const startArray = performance.now();

for (let i = 0; i < N; i++) {
    // unshift() fuerza a V8 a mover los índices de todos los elementos previos
    arregloHistorial.unshift({ id: i, lat: 10.5, lng: -79.2 });
}

const endArray = performance.now();
const timeArray = (endArray - startArray).toFixed(2);
console.log(`[Energía Alta] Arreglo nativo (unshift): ${timeArray} ms`);

// ------------------------------------------------------
// PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
// ------------------------------------------------------
let listaHistorial = new ListaSatelital();
const startList = performance.now();
```

```
28
29  for (let i = 0; i < N; i++) {
30      // prepend() solo reasigna un puntero, sin mover elementos existentes
31      listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
32  }
33
34  const endList = performance.now();
35  const timeList = (endList - startList).toFixed(2);
36  console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);
37
38  // Conclusión de impacto
39  const multiplicador = (timeArray / timeList).toFixed(1);
40  console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces más rápida.`);
41  console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.`);
42
```

Salida inicial de lab04_benchmark.js

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)> node "c:\Users\nicof\OneDri
ve\Desktop\Archivos\Estructura de Datos (Grupal)\APE(Semana4)\lab04_benchmark.js"
[Benchmark] Iniciando inserción de 100000 elementos al INICIO...
[Energía Alta] Arreglo nativo (unshift): 2543.49 ms
[Energía Baja] Lista Enlazada (prepend): 7.67 ms

[Auditoría Verde] La lista enlazada es 331.6 veces más rápida.
Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)>
```

Código final de lab04_benchmark.js

```
1   const { performance } = require('perf_hooks');
2   const { ListaSatelital } = require('./lab04_lista_simple');
3
4   const N = 100000; // 100 mil registros satelitales
5
6   console.log(`[Benchmark] Iniciando inserción de ${N} elementos al INICIO...`);
7
8   // ------------------------------------------------------
9   // PRUEBA 1: ARREGLO NATIVO (Desplazamiento de memoria O(n))
10  // ------------------------------------------------------
11  let arregloHistorial = [];
12  const startArray = performance.now();
13
14  for (let i = 0; i < N; i++) {
15      // unshift() fuerza a V8 a mover los índices de todos los elementos previos
16      arregloHistorial.unshift({ id: i, lat: 10.5, lng: -79.2 });
17  }
18
19  const endArray = performance.now();
20  const timeArray = (endArray - startArray).toFixed(2);
21  console.log(`[Energía Alta] Arreglo nativo (unshift): ${timeArray} ms`);
22
23  // ------------------------------------------------------
24  // PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
25  // ------------------------------------------------------
26  let listaHistorial = new ListaSatelital();
27  const startList = performance.now();
```

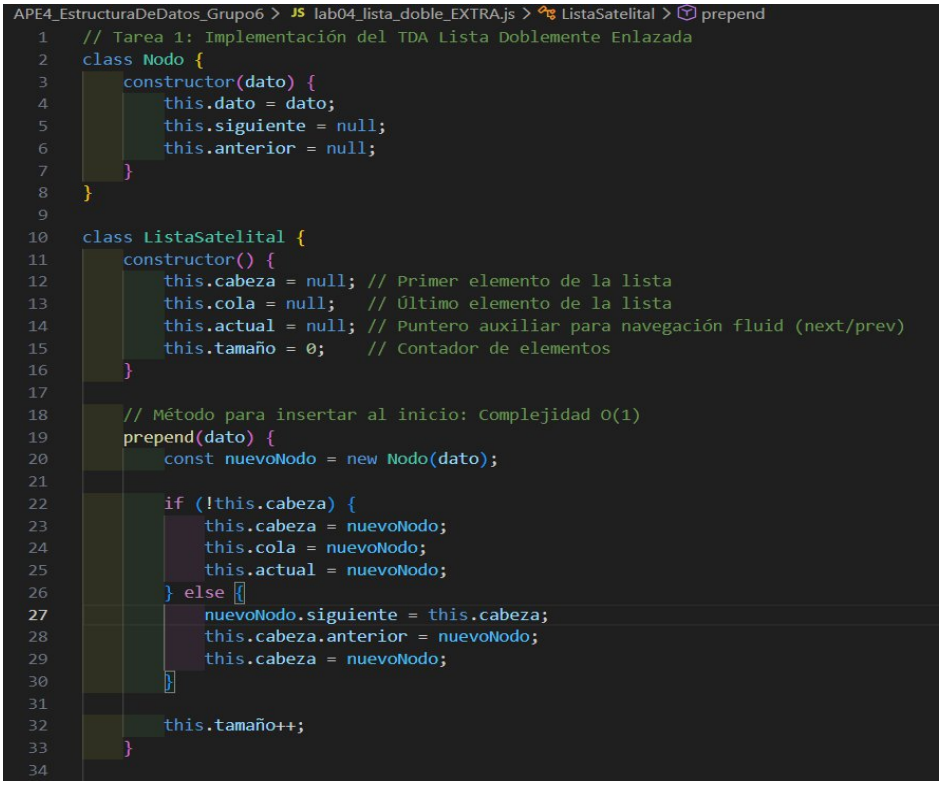
```
28
29  for (let i = 0; i < N; i++) {
30      // prepend() solo reasigna un puntero, sin mover elementos existentes
31      listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
32  }
33
34  const endList = performance.now();
35  const timeList = (endList - startList).toFixed(2);
36  console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);
37
38  // Conclusión de impacto
39  const multiplicador = (timeArray / timeList).toFixed(1);
40  console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces más rápida.`);
41  console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.`);
42
43  listaHistorial.imprimirPrimeros(5);
```

Salida final de lab04_benchmark.js

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)> node "c:\Users\nicof\OneDri
ve\Desktop\Archivos\Estructura de Datos (Grupal)\APE(Semana4)\lab04_benchmark.js"
[Benchmark] Iniciando inserción de 100000 elementos al INICIO...
[Energía Alta] Arreglo nativo (unshift): 2548.22 ms
[Energía Baja] Lista Enlazada (prepend): 6.85 ms

[Auditoría Verde] La lista enlazada es 372.0 veces más rápida.
Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.
[ID: 99999] -> [ID: 99998] -> [ID: 99997] -> [ID: 99996] -> [ID: 99995] -> NULL
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)>
```

Se ha transformado la lista simplemente enlazada en una Lista Doblemente Enlazada e incluir los métodos de navegación next() y prev(), como un extra por parte de la rúbrica.



```
APE4_EstructuraDeDatos_Grupo6 > JS lab04_lista_doble_EXTRA.js > ListaSatelital > prepend
// Tarea 1: Implementación del TDA Lista Doblemente Enlazada
class Nodo {
    constructor(dato) {
        this.dato = dato;
        this.siguiente = null;
        this.anterior = null;
    }
}

class ListaSatelital {
    constructor() {
        this.cabeza = null; // Primer elemento de la lista
        this.cola = null;   // Último elemento de la lista
        this.actual = null; // Puntero auxiliar para navegación fluid (next/prev)
        this.tamaño = 0;    // Contador de elementos
    }

    // Método para insertar al inicio: Complejidad O(1)
    prepend(dato) {
        const nuevoNodo = new Nodo(dato);

        if (!this.cabeza) {
            this.cabeza = nuevoNodo;
            this.cola = nuevoNodo;
            this.actual = nuevoNodo;
        } else {
            nuevoNodo.siguiente = this.cabeza;
            this.cabeza.anterior = nuevoNodo;
            this.cabeza = nuevoNodo;
        }

        this.tamaño++;
    }
```

```
    // --- MÉTODOS REQUERIDOS POR LA RÚBRICA ---

    // Navegación hacia adelante: Complejidad O(1)
    next() {
        if (this.actual && this.actual.siguiente) {
            this.actual = this.actual.siguiente;
            return this.actual.dato;
        }
        return null; // Fin de la lista
    }

    // Navegación hacia atrás: Complejidad O(1)
    prev() {
        if (this.actual && this.actual.anterior) {
            this.actual = this.actual.anterior;
            return this.actual.dato;
        }
        return null; // Inicio de la lista
    }

    // Método para imprimir los primeros elementos (Recorrido secuencial O(n))
    imprimirPrimeros(limite) {
        let nodoActual = this.cabeza;
        let contador = 0;
        let salida = "";

        while (nodoActual && contador < limite) {
            salida += `[ID: ${nodoActual.dato.id}] <-> `;
            nodoActual = nodoActual.siguiente;
            contador++;
        }
        console.log(salida + "NULL");
    }
}

module.exports = { ListaSatelital };
```

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

1. **A diferencia de un arreglo que pre-reserva una cantidad específica de Bytes contiguos, la Lista Enlazada nace como una referencia nula (null). Físicamente en la RAM, ¿de qué está hecha la "conexión" (el campo this.siguiente) que une a dos nodos esparcidos aleatoriamente por el *Heap*?**

   A diferencia de los arreglos contiguos la memoria del heap no es secuencial. Esa “conexión” es una dirección de memoria, la cual guarda el lugar exacto en el heap donde se encuentra el nodo siguiente, de esa manera permitiéndole enlazar bloques de memoria dispersos sin requerir de espacios de memoria contiguos.

2. **Al invocar unshift() en un arreglo, el motor V8 es la causa eficiente que desplaza la memoria. En la operación prepend() de la Lista Enlazada, ¿cuál es el único agente o variable que se debe modificar para lograr la inserción exitosa sin desplazar datos?**

   El único elemento que se va a modificar para lograr dicha inserción es el puntero this.cabeza, ya que el va a pasar a apuntar al nuevo elemento creado, mientras que por su parte el resto de la estructura va a permanecer intacta sin requerir desplazamiento.

3. **Si los arreglos nativos de JavaScript modernos están sumamente optimizados por lenguajes de bajo nivel (C++), ¿cuál es el verdadero telos (propósito final) de tomarse el tiempo para construir manualmente una estructura como la Lista Enlazada en entornos de producción?**

   Aunque los arreglos nativos de JavaScript están altamente optimizados por motores de bajo nivel, cada vez que insertamos un dato al inicio mediante la operación unshift(), la máquina sufre un procesamiento pesado de orden O(n) al verse obligada a desplazar de sitio todos los elementos previos en la memoria. El verdadero propósito (telos) de construir manualmente una lista enlazada en entornos de producción es saltarse ese cuello de botella: al utilizar prepend(), solo reasignamos las referencias de los punteros en tiempo constante de orden O(1), evitando que la CPU desperdicie ciclos de reloj moviendo bloques masivos de memoria. No obstante, desde la perspectiva del Green Computing, existe un intercambio (trade-off) técnico de recursos: si bien logramos una gran eficiencia energética, reducimos la huella térmica del procesador y mitigamos el estrés del hardware, esto conlleva un consumo ligeramente mayor de memoria RAM debido al overhead (sobrecarga) de los punteros adicionales (siguiente y anterior). Este costo adicional de RAM se justifica plenamente en sistemas con alta frecuencia de operaciones, donde priorizar la optimización y el rendimiento energético de la CPU es vital para la sostenibilidad del software.

4. **Un nodo que pierde la referencia (es decir, el nodo anterior ya no apunta a él) sigue existiendo físicamente en los transistores de la memoria RAM con su carga útil intacta, pero la Lista ya no lo reconoce. ¿Ha perdido ese nodo su esencia computacional? ¿Qué sucederá eventualmente con su existencia cuando despierte el *Garbage Collector*?**

Su función lógica si se pierde al desconectarse del recorrido de la estructura, el código ya no tiene ninguna forma de encontrarlo o localizarlo por lo que eventualmente el Garbage Collector va a identificar que ese nodo ya no guarda su referencia y liberará esos bytes de la RAM.

8. Link al repositorio

https://github.com/Jullxn28/Ciclo3_EstructuraDeDatos_Practica.git

9. Link al repositorio

Para el desarrollo de esta actividad se utilizo la ayuda de la IA Gemini, usando el siguiente prompt:

“Dame un resumen claro de los conceptos que debo saber para responder las pre guntas de control de esta guía, necesito la info base para yo mismo armar las res puestas”

1. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	